

1. Title - HEAP SORT

Implementation and Performance Analysis of Heap Sort Algorithm using Binary Heap Data Structure

2. Abstract

This mini project focuses on the study and implementation of the Heap Sort Algorithm, a classical algorithm in the domain of sorting techniques. The objective of this project is to understand the design principles of Heap Sort, analyze its computational complexity, implement it in C++, and evaluate its performance on various inputs including edge cases and extreme cases.

The project demonstrates the working of Heap Sort through modular code with detailed inline comments. The algorithm utilizes the Binary Heap Data Structure, specifically a Max Heap, for efficient execution and optimization.

Time complexity is analyzed as:

- Best Case: $O(n \log n)$
- Average Case: $O(n \log n)$
- Worst Case: $O(n \log n)$

The space complexity of the algorithm is determined to be $O(1)$ for in-place implementation. Amortized analysis is carried out where applicable.

Possibilities for optimization using advanced data structures such as Fibonacci Heaps and Balanced Binary Search Trees are also explored.

Practical applications of the algorithm in areas such as Operating System Scheduling and Database Management Systems are discussed, with special relevance to Artificial Intelligence and Machine Learning highlighted.

3. Introduction

3.1 Background and Motivation

Algorithms are important in computer science because they provide step-by-step methods to solve problems efficiently. Sorting is one of the most common operations used in data processing, searching, databases, and many software applications. Efficient sorting helps organize data properly and improves the speed of data retrieval and processing.

Heap Sort is a comparison-based sorting algorithm that uses the Binary Heap Data Structure to sort elements efficiently. It is widely used because it provides $O(n \log n)$ time complexity in best, average, and worst cases. Heap Sort also performs sorting in-place, which means it requires very little extra memory.

Heap Sort was introduced by J. W. J. Williams in 1964. It belongs to the class of tree-based sorting algorithms and is used in applications such as Priority Queues, Operating System Scheduling, and Database Systems. The algorithm works by repeatedly removing the largest element from the heap and placing it in the correct sorted position. Due to its efficiency and reliable performance, Heap Sort is an important algorithm in computer science.

3.2 Problem Statement

Problem Statement:

Given an unsorted list of elements, design and implement the Heap Sort Algorithm to sort the elements in ascending order using the Binary Heap Data Structure.

Input:

- An array of integers entered by the user
- The number of elements in the array

Output:

The sorted array in ascending order

Constraints:

- Number of elements $n \geq 1$
- Array elements can be positive or negative integers
- The input size should fit within available memory
- Duplicate values are allowed

3.3 Objectives

- To understand the design and working principle of the Heap Sort Algorithm.
- To implement the algorithm in C++ with well-commented modular code.
- To analyze time and space complexity in best, average, and worst cases.
- To study the efficiency of Heap Sort using the Binary Heap Data Structure.
- To test the algorithm on normal inputs, boundary cases, and extreme cases.
- To explore possible optimizations and improvements in sorting performance.
- To study real-world applications of Heap Sort in computer science, Artificial Intelligence, and Machine Learning.

3.4 Scope of the Project

This project focuses on the implementation and analysis of the Heap Sort Algorithm using the Binary Heap Data Structure.

This project includes algorithm design, working procedure, complexity analysis, and testing with different types of input data. It also explains the applications and advantages of Heap Sort in various computer science fields.

4. Implementation with Comments

4.1 Programming Language and Environment

Language	<i>C Programming</i>
IDE / Editor/Compiler	<i>Visual Studio Code (VS Code) / GCC Compiler</i>
OS / Platform	<i>Linux EndeavourOS</i>
Libraries Used	<i>stdio.h and stdlib.h</i>
Github Link	<i>https://github.com/yashwanthappdev/heap-sort-project</i>

4.2 Algorithm Design

ALGORITHM *Heap sort*

INPUT

- Array of integer elements
- Number of elements in the array

OUTPUT

- Array sorted in ascending order

FUNCTION *heapSort(array, size)*

FOR $i = \text{size} / 2 - 1$ **DOWNTO** 0

CALL *heapify(array, size, i)*

END FOR

FOR $i = \text{size} - 1$ **DOWNTO** 1

SWAP $\text{array}[0]$ with $\text{array}[i]$

CALL *heapify(array, i, 0)*

END FOR

END FUNCTION

FUNCTION *heapify(array, size, root)*

$\text{largest} \leftarrow \text{root}$

$\text{left} \leftarrow 2 \times \text{root} + 1$

$\text{right} \leftarrow 2 \times \text{root} + 2$

IF $\text{left} < \text{size}$ **AND** $\text{array}[\text{left}] > \text{array}[\text{largest}]$

$\text{largest} \leftarrow \text{left}$

END IF

IF $\text{right} < \text{size}$ **AND** $\text{array}[\text{right}] > \text{array}[\text{largest}]$

$\text{largest} \leftarrow \text{right}$

END IF

IF $\text{largest} \neq \text{root}$

SWAP $\text{array}[\text{root}]$ and $\text{array}[\text{largest}]$

CALL *heapify(array, size, largest)*

END IF

END FUNCTION

4.3 Source Code with Inline Comments

```

/* =====

* Mini Project: Implementation and Performance Analysis of Heap
Sort Algorithm using Binary Heap Data Structure

* Algorithm      : Heap Sort

* Author         : YASHWANTH P | R24EJ162

* Date          : 18 - 05 - 2026

* Description    : Program to sort elements using Heap Sort
Algorithm

===== */

```

```
/* --- Include Necessary Header Files --- */
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
/* --- Constant Definition --- */
```

```
#define MAX_SIZE 100
```

```
/* =====  
Function Name : heapify  
Purpose : Maintains Max Heap property for the subtree  
Input : Integer array, heap size, root index  
Output : No return value  
===== */  
  
void heapify(int arr[], int size, int root) {  
    /* Assume root as largest element */  
    int largest = root;  
  
    /* Calculate left and right child indexes */  
    int left = 2 * root + 1;  
    int right = 2 * root + 2;  
  
    /* Check if left child is larger than root */  
    if (left < size && arr[left] > arr[largest]) {  
        largest = left;  
    }  
  
    /* Check if right child is larger than largest */  
    if (right < size && arr[right] > arr[largest]) {  
        largest = right;  
    }  
  
    /* If largest element is not root */  
    if (largest != root) {  
        /* Swap root with largest element */  
        int temp = arr[root];  
        arr[root] = arr[largest];  
        arr[largest] = temp;  
  
        /* Recursively heapify affected subtree */  
        heapify(arr, size, largest);  
    }  
}
```

```
/* =====  
Function Name : heapSort  
Purpose : Sorts array elements using Heap Sort  
Input : Integer array and size of array  
Output : Sorted array  
===== */  
  
void heapSort(int arr[], int size) {  
    int i;  
  
    /* Build Max Heap */  
    for (i = size / 2 - 1; i >= 0; i--) {  
        heapify(arr, size, i);  
    }  
  
    /* Extract elements one by one from heap */  
    for (i = size - 1; i > 0; i--) {  
        /* Swap current root with last element */  
        int temp = arr[0];  
        arr[0] = arr[i];  
        arr[i] = temp;  
  
        heapify(arr, i, 0);  
    }  
}  
  
/* =====  
Function Name : printArray  
Purpose : Displays array elements  
Input : Integer array and size  
Output : Prints array elements  
===== */  
  
void printArray(int arr[], int size) {  
    int i;  
  
    for (i = 0; i < size; i++) {  
        printf("%d ", arr[i]);  
    }  
  
    printf("\n");  
}
```

```
/* =====  
Main Function  
Purpose : Accept input, perform Heap Sort, display output  
===== */  
  
int main() {  
    int arr[MAX_SIZE];  
    int size, i;  
  
    /* Read number of elements */  
    printf("Enter number of elements: ");  
    scanf("%d", &size);  
  
    /* Read array elements */  
    printf("Enter array elements:\n");  
  
    for (i = 0; i < size; i++) {  
        scanf("%d", &arr[i]);  
    }  
  
    /* Display original array */  
    printf("\nOriginal Array:\n");  
    printArray(arr, size);  
  
    /* Perform Heap Sort */  
    heapSort(arr, size);  
  
    /* Display sorted array */  
    printf("\nSorted Array:\n");  
    printArray(arr, size);  
  
    return 0;  
}
```


5. Modules Identified

The implementation has been divided into the following logical modules for clarity, reusability, and ease of testing:

Sl. no	Module Name	Description	Function(s)
1	Input Module	Reads array size and elements from the user.	main()
2	Heapify Module	Maintains the Max Heap property for the subtree.	heapify()
3	Heap Sort Module	Builds the heap and performs the sorting process.	heapSort()
4	Output Module	Displays the original and sorted array elements.	printArray()
5	Main Module	Controls overall program execution and function calls.	main()

6. Output with Screenshots

6.1 Test Case Strategy

All test cases have been categorized into:

Normal Cases – Typical input that the algorithm is designed to handle.

Boundary / Edge Cases – Minimum or maximum valid input (e.g., empty input, single element, duplicates).

Extreme Cases – Very large inputs, negative values, all-same elements, or already-sorted arrays.

6.2 Test Case Table

TC	Test Category	Input	Expected Output	Actual Output	Status
01	Normal	[5; 3; 8; 1]	[1; 3; 5; 8]	[1; 3; 5; 8]	PASS
02	Normal	[10; 20; 15]	[10; 15; 20]	[10; 15; 20]	PASS
03	Edge – Empty	[]	[]	[]	PASS
04	Edge – Single	[42]	[42]	[42]	PASS
05	Edge – Duplicates	[5; 5; 5]	[5; 5; 5]	[5; 5; 5]	PASS
06	Extreme – Large n	[10 Random Values]	Sorted list	Sorted list	PASS
07	Extreme – Reverse	[9; 8; 7; 6]	[6; 7; 8; 9]	[6; 7; 8; 9]	PASS
08	Extreme – Negatives	[-3; -1; -7]	[-7; -3; -1]	[-7; -3; -1]	PASS

6.3 Screenshots of Output

```
Enter number of elements: 4
Enter array elements:
5 3 8 1

Original Array:
5 3 8 1

Sorted Array:
1 3 5 8
```

Screenshot 6.3.1 – Normal Case Output

```
Enter number of elements: 3
Enter array elements:
10 20 15

Original Array:
10 20 15

Sorted Array:
10 15 20
```

Screenshot 6.3.2 – Normal Case Output

```
Enter number of elements: 1
Enter array elements:
42

Original Array:
42

Sorted Array:
42
```

Screenshot 6.3.3 – Edge - Single Output

```
Enter number of elements: 5
Enter array elements:
5 5 5 5 5
```

```
Original Array:
5 5 5 5 5
```

```
Sorted Array:
5 5 5 5 5
```

Screenshot 6.3.4 – Edge – Duplicates Output

```
Enter number of elements: 4
Enter array elements:
9 8 7 6
```

```
Original Array:
9 8 7 6
```

```
Sorted Array:
6 7 8 9
```

Screenshot 6.3.5 – Extreme Reverse Output

```
Enter number of elements: 3
Enter array elements:
-3 -1 -7
```

```
Original Array:
-3 -1 -7
```

```
Sorted Array:
-7 -3 -1
```

Screenshot 6.3.5 – Extreme Negative Output

```
Enter number of elements: 4
Enter array elements:
1 2 3 4

Original Array:
1 2 3 4

Sorted Array:
1 2 3 4
```

Screenshot 6.3.5 – Extreme Sorted Output

```
Enter number of elements: 10
Enter array elements:
45 12 78 34 23 89 11 67 90 2

Original Array:
45 12 78 34 23 89 11 67 90 2

Sorted Array:
2 11 12 23 34 45 67 78 89 90
```

Screenshot 6.3.5 – Extreme Large - n Output

7. Time and Space Complexity Analysis

7.1 Time Complexity

Metric	Best Case	Average Case	Worst Case
Time Complexity	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Space Complexity	$O(1)$	$O(1)$	$O(1)$

7.2 Detailed Analysis

Best Case – $O(n \log n)$

The best case occurs when the array is already arranged in a way that satisfies the heap property. Even in this case, Heap Sort must build the heap and repeatedly perform heapify operations while extracting elements. Therefore, the time complexity remains $O(n \log n)$.

Average Case – $O(n \log n)$

In the average case, Heap Sort performs heap construction followed by repeated heapify operations for each element. Since each heapify operation takes $O(\log n)$ time and it is performed for all elements, the average time complexity is $O(n \log n)$.

Worst Case – $O(n \log n)$

The worst case occurs when maximum heap adjustments are required during every heapify operation. Even in this situation, Heap Sort performs consistently with $O(n \log n)$ time complexity because each heapify operation takes logarithmic time and is repeated for all elements.

7.3 Space Complexity

In-place - Heap Sort uses $O(1)$ additional space apart from the input array.

Recursive Call Stack - The recursive heapify() function uses stack space of $O(\log n)$ due to recursive function calls.

Auxiliary Space - No additional data structures are required, making Heap Sort memory efficient compared to algorithms like Merge Sort.

7.4 Amortized Complexity Analysis

Amortized analysis provides the average cost per operation over a sequence of operations, rather than analyzing each operation in isolation.

Aggregate Method:

The total cost of building the heap and performing heapify operations for all n elements is $O(n \log n)$. Since heapify operations are repeatedly applied during sorting, the average cost per operation remains logarithmic.

Total Cost for n operations:

$$T(n) = O(n \log n)$$

Amortized Cost per operation:

$$T(n) / n = O(\log n)$$

Thus, the amortized cost of each heapify operation in Heap Sort is $O(\log n)$.

8. Can the Algorithm Be Optimized?

Yes – Heap Sort can be optimized in terms of practical performance and efficiency for specific use cases. Although the theoretical time complexity already remains $O(n \log n)$ in all cases, certain improvements can reduce execution overhead and improve memory or cache performance.

8.1 Current Limitations

- Heap Sort is not a stable sorting algorithm, meaning equal elements may not preserve their original order.
- Heap Sort has lower cache efficiency because heap elements are accessed non-sequentially in memory.
- Recursive heapify() calls may increase function call overhead for large inputs.
- Heap Sort may perform slower than Quick Sort in practical applications despite having the same average complexity.

8.2 Optimization Strategies

Optimization Strategy	Data Structure Used	Improvement Achieved
Use iterative heapify instead of recursion	Iterative approach	Reduces recursion overhead
Use optimized Binary Heap structure	Binary Heap	Faster heap operations
Cache-friendly memory access	Array optimization	Improves execution speed
Hybrid sorting approach	Heap Sort + Insertion Sort	Better performance for small arrays
Parallel Heap Construction	Parallel Processing	Faster execution on large datasets

One important optimization for Heap Sort is replacing recursive heapify() with an iterative implementation. This reduces recursive function call overhead and improves execution speed for large arrays.

Before Optimization

- Recursive heapify uses additional recursive stack calls.
- Practical execution may become slower for very large datasets.

After Optimization

- Iterative heapify removes recursion overhead.
- Memory usage becomes more efficient.
- Faster execution in practical systems.

Complexity Comparison

Operation	Before Optimization	After Optimization
Time Complexity	$O(n \log n)$	$O(n \log n)$
Auxiliary Space	$O(\log n)$	$O(1)$

9. Applications of the Algorithm

9.1 Applications in Computer Science

1 Operating System Scheduling – Heap Sort is used in priority-based CPU scheduling and task management.

2 Priority Queue Implementation – Binary Heaps used in Heap Sort are widely used to implement priority queues efficiently.

3 Database Management Systems – Used for efficient sorting of records and query processing.

4 Network Routing Algorithms – Heap structures help in shortest path algorithms such as Dijkstra's Algorithm.

5 Event-Driven Simulations – Helps managing event based on priority and execution time.

6 Data Processing Applications – Useful in large-scale data sorting and processing systems.

9.2 Applications in Artificial Intelligence (AI)

1 Priority-Based Search Algorithms – Heap structures are used in AI search algorithms like A* Search for selecting optimal nodes.

2 Task Scheduling in Intelligent Systems – AI systems use priority queues for efficient task execution and resource allocation.

3 Real-Time Decision Systems – Heap-based processing helps AI systems quickly process high-priority tasks.

4 AI Game Development – Heap structures are used in game AI for event handling, path finding, and move prioritization.

9.3 Applications in Machine Learning (ML)

1 Data Preprocessing – Heap Sort is used for sorting datasets before training machine learning models.

2 Feature Ranking and Selection – It Help to identify top important features efficiently.

3 Large Dataset Management – Heap-based sorting is useful when handling large-scale training and testing datasets.

10. Conclusion

In this mini project, we successfully designed, implemented, and analyzed the Heap Sort Algorithm as part of the Design and Analysis of Algorithms course. The implementation was carried out in C programming language using well-structured modular code with detailed inline comments for better readability and understanding.

Through this project, we gained a clear understanding of the working mechanism of Heap Sort and the Binary Heap Data Structure. Heap Sort follows a comparison-based and tree-based sorting approach to efficiently sort elements. The time complexity of Heap Sort was determined to be $O(n \log n)$ in the best, average, and worst cases. The space complexity was found to be $O(1)$ for in-place implementation. Amortized analysis was not significantly applicable because Heap Sort is a one-time sorting algorithm and does not involve repeated dynamic operations.

The testing phase confirmed the correctness of the algorithm for normal inputs, boundary cases, duplicate elements, negative values, and reverse sorted arrays. We also explored optimization techniques such as iterative heapify implementation, which can reduce recursion overhead and improve practical performance.

Heap Sort has important applications in Operating System Scheduling, Priority Queues, Database Systems, Artificial Intelligence, and Machine Learning tasks such as data preprocessing and priority-based processing. Overall, this project helped in understanding the importance of efficient algorithms for developing scalable and optimized software systems.

10.1 Future Enhancements

Implement an iterative version of heapify to reduce recursion overhead.

Develop a graphical visualization tool for Heap Sort for educational purposes.

Implement parallel Heap Sort for handling very large datasets efficiently.

Compare Heap Sort performance with Quick Sort and Merge Sort using large datasets.

Extend the implementation to support sorting of different data types such as strings and floating-point values.

11. References

T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.

S. Dasgupta, C. H. Papadimitriou, and U. Vazirani, *Algorithms*. New York, NY, USA: McGraw-Hill Education, 2006.

A. V. Aho, J. E. Hopcroft, and J. D. Ullman, *The Design and Analysis of Computer Algorithms*. Reading, MA, USA: Addison-Wesley, 1974.

J. W. J. Williams, “**Algorithm 232: Heapsort**,” *Communications of the ACM*, vol. 7, no. 6, pp. 347–348, 1964.

12. Declaration

We, the undersigned student, hereby declare that the mini project report the titled “**Implementation and Performance Analysis of Heap Sort Algorithm using Binary Heap Data Structure**” submitted to the Department of **Computer Science and Engineering** in partial fulfillment of the requirements for the **4th Semester Design and Analysis of Algorithms** course is our original work. The project has been carried out by us under the guidance of **Introduction to Algorithms, 4th ed. Cambridge, MA, USA: MIT Press, 2022** and has not been submitted elsewhere for any other examination or award.

Student Name	Yashwanth P
Student Signature	
Student SRN	R24EJ162
Faculty	Prof. DivyaShree / Dr. Aravindhan
Faculty Signature	
Date	18 - 05 - 2026